

LAB PRACTICUM 01

Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

Student: Symbat Aldanova
Student ID: 230103170
Hardware: MacBook Pro, Apple M4
Operating System: macOS 26.5.2 (arm64)

Task 1 - Host Silicon Audit & Flynn Taxonomy

Property	Measured / Reported Value
Machine	MacBook Pro (Mac16,1)
Processor / SoC	Apple M4
CPU cores	10 total: 4 Performance + 6 Efficiency
Physical / logical cores	10 / 10
Memory	16 GB
Architecture	arm64
L1 data cache	64 KB
L1 instruction cache	128 KB
L2 cache	4 MB
L3 cache	Not reported by macOS sysctl
SIMD / vector support	ARM NEON / Advanced SIMD; SME and SME2 reported
Base / max clock	Not reported by macOS sysctl

Q1.1 - Flynn taxonomy. SISD describes sequential execution with one instruction stream operating on one data stream; a single CPU core running a sequential program is an example. SIMD uses one instruction to process multiple data elements at once; the Apple M4 reports ARM NEON / Advanced SIMD support. MIMD allows multiple cores to execute different instruction streams on different data simultaneously; the 10-core M4 can run multiple processes or threads in parallel.

Task 2 - Multi-Thread Scaling Benchmark

A fixed compute-bound workload of 20,000,000 iterations was divided among 1, 2, 4, 8, 16, and 32 Python worker processes. Each configuration was measured three times. The table uses the arithmetic mean of the three measured runs.

Workers	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg (s)	Speedup	Efficiency
1	0.8421	0.8316	0.8165	0.8301	1.00x	100.00%
2	0.4850	0.4321	0.4671	0.4614	1.80x	89.95%
4	0.2563	0.2403	0.2557	0.2508	3.31x	82.75%
8	0.1994	0.2007	0.1931	0.1977	4.20x	52.48%
16	0.2302	0.2311	0.2243	0.2285	3.63x	22.70%
32	0.3328	0.3183	0.3154	0.3222	2.58x	8.05%

Q2.1 - Saturation point. The best measured time was 0.1977 s at 8 workers, corresponding to 4.20x speedup. Performance degraded at 16 workers (0.2285 s) and 32 workers (0.3222 s). Therefore, the observed saturation point for this experiment was approximately 8 workers. The machine has 10 CPU cores, so 16 and 32 workers oversubscribe the available CPU resources and introduce additional process scheduling and coordination overhead.

Task 3 - Unsynchronized Shared Counter & Race Condition

Ten native Java threads each incremented the same shared counter 1,000,000 times without a lock. The expected final value was 10,000,000. The experiment was executed ten consecutive times.

Run	Actual	Lost updates	Time (s)
1	1,299,302	8,700,698	0.0232
2	7,869,337	2,130,663	0.0007
3	10,000,000	0	0.0005
4	10,000,000	0	0.0004
5	10,000,000	0	0.0004
6	10,000,000	0	0.0003
7	10,000,000	0	0.0004
8	10,000,000	0	0.0003
9	10,000,000	0	0.0003
10	10,000,000	0	0.0003

Q3.1 - Why the race occurs. The expression `counter++` is a read-modify-write operation rather than one indivisible atomic action. Two threads can read the same old value, calculate the same incremented value, and then both write it back. One increment is therefore lost. Runs 1 and 2 demonstrate this directly. The fact that later runs happened to reach the expected value does not prove the code is safe; race conditions are nondeterministic.

Locked version

Run	Actual	Time (s)
1	10,000,000	0.1859
2	10,000,000	0.2212
3	10,000,000	0.0189
4	10,000,000	0.0183
5	10,000,000	0.0178
6	10,000,000	0.0192
7	10,000,000	0.0192
8	10,000,000	0.0182
9	10,000,000	0.0207
10	10,000,000	0.0190

With synchronized locking, all ten runs produced the correct value of 10,000,000. Synchronization prevents simultaneous unsafe updates, but it adds contention and serialization overhead because threads must coordinate access to the same shared counter. Java JIT warm-up also affected the first runs, so individual early timing values should not be treated as a clean synchronization-only comparison.

Task 4 - Empirical Amdahl vs Gustafson

Using the measured Task 2 averages $T_1 = 0.8301$ s and $T_2 = 0.4614$ s, the estimated parallel fraction and scaling predictions are:

Metric	Result
Estimated parallel fraction p	0.8883 (88.83%)
Amdahl maximum speedup	8.95x

Metric	Result
Amdahl speedup at 64 processors	7.96x
Gustafson speedup at 64 processors	56.96x

Interpretation. Amdahl's Law assumes a fixed total workload, so the remaining serial fraction places a hard limit on speedup; for the fitted fraction, the theoretical maximum is about 8.95x. Gustafson's Law assumes that the problem size can grow with the number of processors, so the estimated scaled speedup at 64 processors is much larger (56.96x). This illustrates the difference between strong scaling of a fixed problem and scaled/weak-style reasoning where more processors are used to solve a larger problem.

Conclusion

The experiments show that additional parallel workers improve performance only up to a hardware- and workload-dependent point. On this Apple M4 system, the compute benchmark reached its best measured time at 8 workers and then slowed under oversubscription. The shared-counter experiment demonstrated that parallel execution can also introduce correctness problems: unsynchronized updates lost millions of increments in some runs, while synchronized access restored correctness at the cost of coordination overhead. The Amdahl and Gustafson calculations further show why fixed-size and scaled-size parallel workloads can produce very different expectations for speedup.